

学号： 2106410119



沈阳建筑大学

大数据技术基础 作业

2023 年秋季学期

学 院： 计算机科学与工程学院

专业班级： 计算机 2101 班

姓 名： 陈芷涵

第五次作业

抓取一个公开网站上的数据，数据必须是多页，存为 csv 格式；如果数据是多个 pdf，将其合并为一个 pdf。程序要尽量自动化，减少人为干预。

代码：

```
import requests
import json
import re
import pprint
import csv
from time import sleep
from random import randint

keyword = input('请输入你想要搜索的岗位： ')
pages = input('请输入你想要爬取的页数： ')
with open(f'{keyword}zhilian.csv','w',encoding='utf-8',newline='') as filename:
    dictwriter = csv.DictWriter(filename,fieldnames=[
        '岗位名称',
        '公司名称',
        '学历',
        '公司规模',
        '地区',
        '福利待遇',
        '行业',
        '薪资',
        '经验',
        '发布时间',
        '详情页',
    ])
    dictwriter.writeheader()
    for page in range(1,int(pages)):
        sleep(randint(3, 8))
        print(f'=====正在采集第{page}页的数据内容=====')
        url = f'https://sou.zhaopin.com/?jl=765&kw={keyword}&p={pages}'
        headers = {
            #输入网址上的 headers 即可
        }
        response = requests.get(url=url,headers=headers)

        html_data = response.text

        re.findall('locationInfo":{},"selectCity":"","positionList":(. *?), "isSupportBatchDelivery":true',response.text)[0]

        json_data = json.loads(html_data)
```

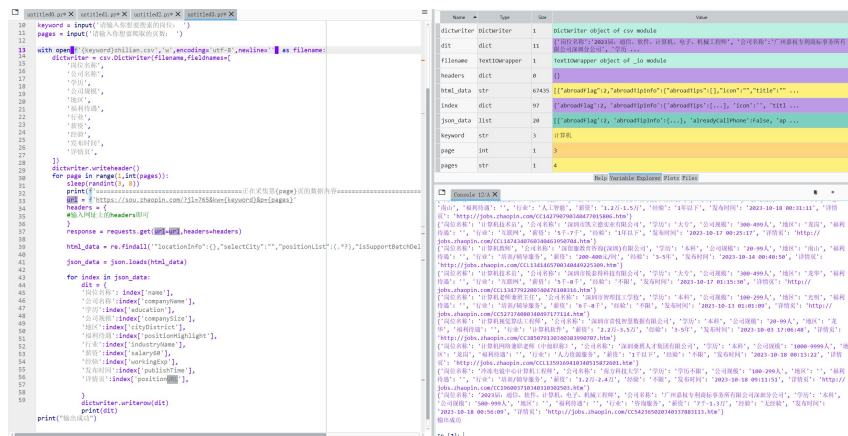
for index in json_data:

```
dit = {
'岗位名称': index['name'],
'公司名称': index['companyName'],
'学历': index['education'],
'公司规模': index['companySize'],
'地区': index['cityDistrict'],
'福利待遇': index['positionHighlight'],
'行业': index['industryName'],
'薪资': index['salary60'],
'经验': index['workingExp'],
'发布时间': index['publishTime'],
'详情页': index['positionURL'],
}
dictwriter.writerow(dit)
print(dit)
```

程序分析：

1. 导入必要的库
2. 获取用户输入： 程序首先要求用户输入要搜索的岗位关键词和要爬取的页数。
3. 打开 CSV 文件
4. 循环爬取多页数据
5. 添加随机延迟
6. 构建爬取 URL
7. 发送 HTTP 请求并获取页面内容：
8. 解析 HTML 数据
9. 将提取的 JSON 数据解析为 Python 字典：使用`json.loads`解析 JSON 格式的数据，将其转换为 Python 字典
10. 遍历招聘信息：
遍历 JSON 数据中的每个招聘信息，将信息提取到一个字典（`dit`）中
11. 将信息写入 CSV 文件：
12. 打印招聘信息：

运行截图：



第六次作业 page 119 6.1

葡萄酒数据集 (wine.data) 搜集了法国不同产区葡萄酒的化学指标。试建立决策树、随机森林和 xgBoost3 种分类器模型，比较各种分类器在此数据集上的效果。

【提示】：每种分类器，需要对参数进行尝试，找出此种分类算法的较优模型，再与其他分类器性能进行比较。

代码：

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

# 读取数据集
data = pd.read_csv('wine.data', header=None)
# 将数据集分为特征和标签
X = data.iloc[:, 1:]
y = data.iloc[:, 0]

from sklearn.preprocessing import LabelEncoder
#XGBoost 要求目标变量必须是从 0 开始的整数，而 y 变量取值是 [1, 2, 3], 使用
LabelEncoder 进行转换
le = LabelEncoder()
xgy = le.fit_transform(y)
# 将数据集分为训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
X_train, X_test, xgy_train, xgy_test = train_test_split(X, xgy, test_size=0.3, random_state=42)
# 建立决策树模型
dtc = DecisionTreeClassifier(random_state=42)
dtc.fit(X_train, y_train)
dtc_score = dtc.score(X_test, y_test)
# 建立随机森林模型
rfc = RandomForestClassifier(random_state=42)
rfc.fit(X_train, y_train)
rfc_score = rfc.score(X_test, y_test)
# 建立 xgBoost 模型
xgb = XGBClassifier(random_state=42)
xgb.fit(X_train, xgy_train)
xgb_score = xgb.score(X_test, xgy_test)
# 输出各个模型在测试集上的表现
print('Decision Tree Classifier score:', dtc_score)
print('Random Forest Classifier score:', rfc_score)
print('XGBoost Classifier score:', xgb_score)
```

程序说明：

从 wine.data 中读取数据将数据集中除第一列外的所有列赋值给变量 x，将数据集中第一列

赋值给变量 y 。使用 `LabelEncoder` 进行对 y 的值进行转换，将数据集分为训练集和测试集，`test_size` 为 0.3, `random_state` 为 42, 分别建立三种模型，并对模型的性能进行测试，输出各个模型在测试集上的表现，发现随机森林的模型最好

运行截图

